GitHub Actions (CI/CD)

→ **GitHub Actions**

It is a Build-in Github automation & CI/CD platform that automatically executes tasks when specific events occur in a repository.

↳ **It can Automate-**

- Running Python tests.
- Building applications & Docker Images.
- checking code quality.
- Deploying applications & API's.
- Running scheduled ML experiments.

→ **Git Vs GitHub Vs GitHub Actions**

Technology	Purpose
(1) Git	Track code changes & manages version history.
(2) GitHub	Hosts repositories & supports collaboration.
(3) GitHub Actions	Automates testing, Building & Deployment.

→ Understanding CI/CD

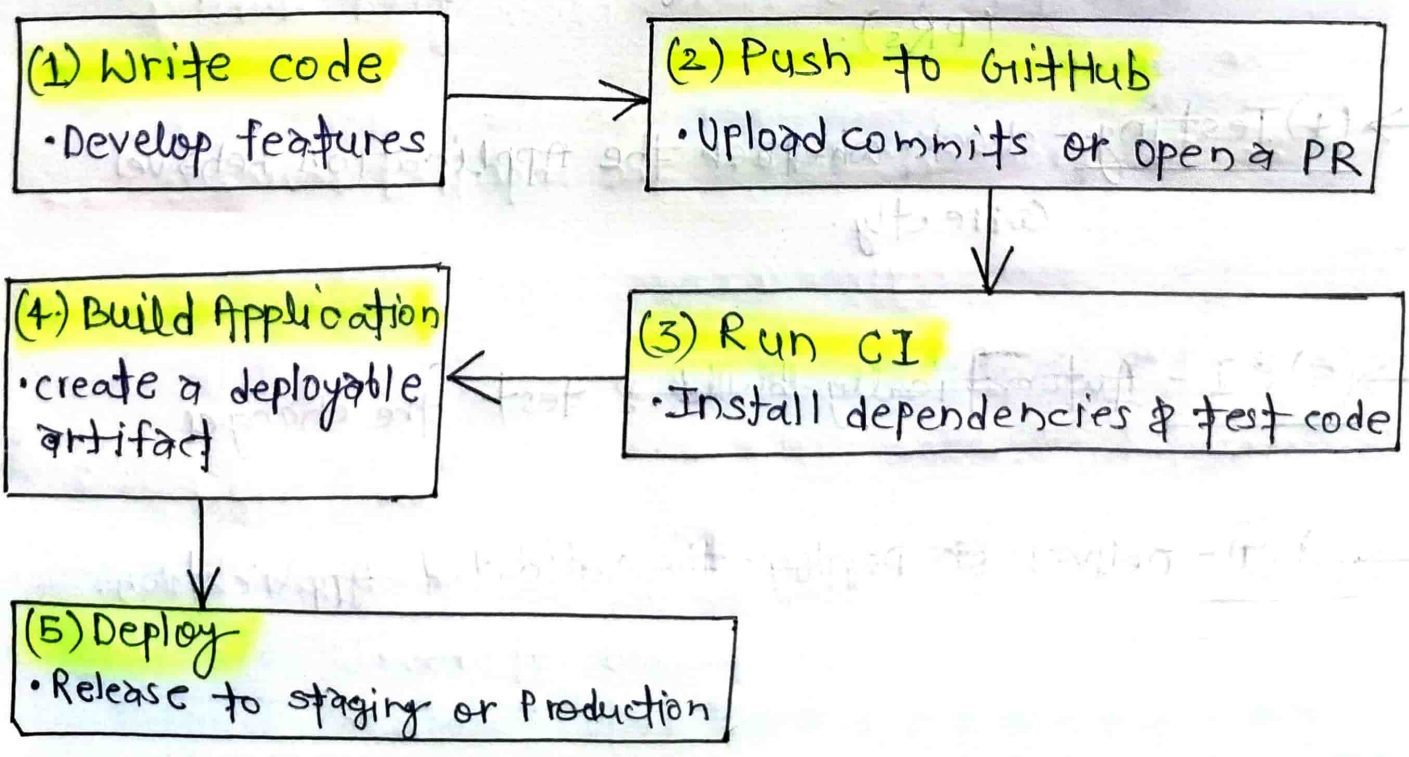
↳ **CI (continuous Integration)** - Automatically builds and tests code whenever developers push changes or open pull requests.

- Goal - Detect bugs early & keep code working.

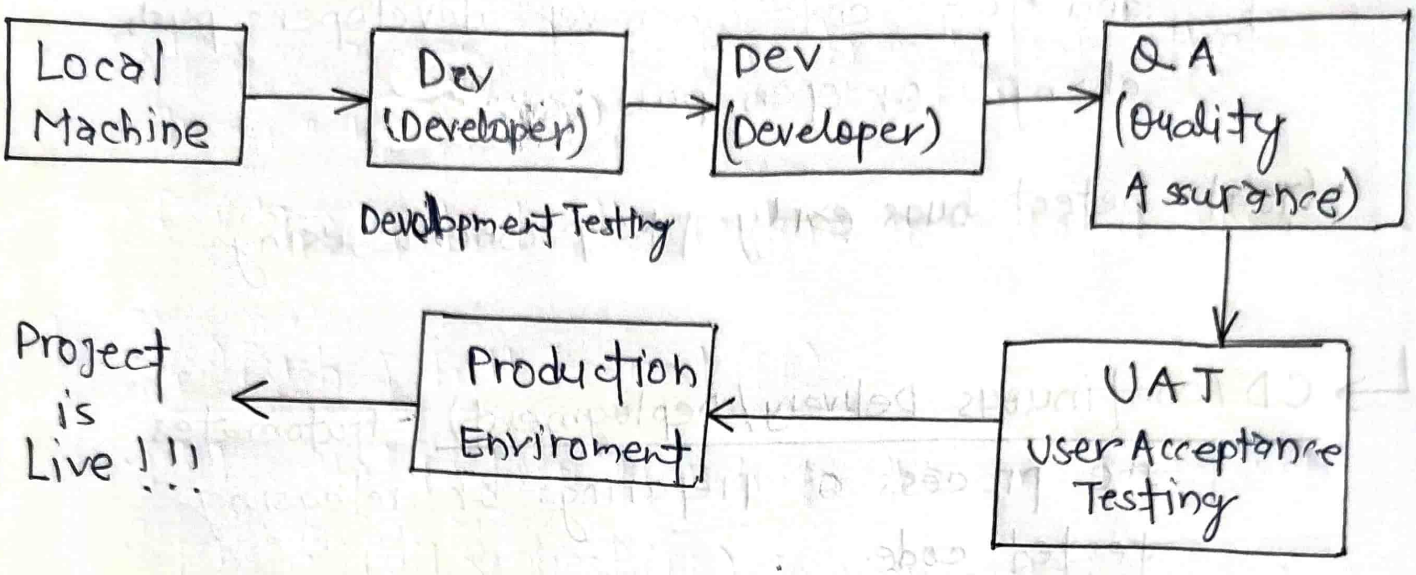
↳ **CD (continuous Delivery / Deployment)** - Automates the process of preparing or releasing tested code.

- **continuous Delivery** - code is ready for release, often requiring approval for production.
- **continuous Deployment** - Every qualifying change is automatically release to production.

→ CI/CD Pipeline



→ Code-to-Production Pipeline / software Deployment flow Diagram



→ Development workflow stages

↳ (1). Coding - writing application code in an IDE like VsCode.

↳ (2) Version control - Track changes using Git & branches

↳ (3) Code Review - Review changes through pull requests (PRs).

↳ (4) Testing - check whether the Application behaves correctly.

↳ (5) CI - Automatically build & test the changes

↳ (6) CD - Deliver or Deploy the validated Application.

→ Types of Testing

Test	What it checks
(1) Unit Testing	Individual functions or components.
(2) Integration Testing	Whether multiple modules work together.
(3) End-to-End testing	Whether the complete workflow works correctly.

→ Deployment enviroments

- **Development (Dev)** - Initial development & Testing
- **Quality Assurance (QA)** - Functional & quality testing
- **User Acceptance Testing (UAT)** - Final validation by users or stakeholders.
- **Production** - The live Application used by real users.

Note- For collaborative project where multiple of developer is contributing. so they don't use & make changes code main branch /sourcecode they make connect new branch from branch. If everything is perfect by all then changes applied.

→ Developer Workflow

↳ (1.) Feature Development

- Developer "A" works on a specific feature or task.
- Creates a seperate feature branch from the "main" branch.
- Developes & tests the feature in a Local IDE.

↳ (2.) Push & Pull Request (PR)

- Pushes the feature branch to the remote repository.
- Opens a Pull Request (PR) to merge changes into "main" branch.
- Team members review the PR for code quality, style & errors.

↳ (3.) Code Review

- Reviewers examine the code & suggest improvements.
- The developer fixes issues identified during the review.
- The PR proceeds when the required reviews are completed.

↳ (4.) Automated CI Pipeline

- Opening a PR automatically triggers the CI Pipeline.
- CI builds the Application & runs automated tests.
- If all checks pass & required approvals are obtained the PR can be merged into "main".

(5.) Continuous Deployment (CD)

- Merging the PR can trigger the CD pipeline.
- The Application is Automatically deployed to the development or staging enviroment.
- The After validation, it can be released to the production enviroment.

Benefits of Developer Workflow

- **Improved collaboration** - clarifies team responsibility.
- **Higher Code Quality** - code reviews & automated tests improve Quality.
- **Reduced Errors** - Automation minimizes human mistakes.
- **Faster Delivery** - speeds up Development & realease.
- **Continuous Feedback** - Helps identify & fix issues Quickly.

→ GitHub Action - Hands-on CI Project

↳ (1.) Project setup & Git Intialization

- create an empty Github repo named "app-github-action".
- open Vs Code & create a readme file for this project in project folder.

```
echo "This is the Python App, code-Aniket" > readme.md
git init
git add
git commit -m "first commit"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

Key Terms:-

- **git init** = Initializes a local Git repository
- **git add** = stages all changes.
- **git commit** = saves a snapshot of charges.
- **git branch -M main** = Rename the current branch to main.
- **git remote add origin** = connects the local repository to Github.
- **git push -u origin main** = Pushes commits & sets the upstream branch.

(2.) Dependencies & source code

make "requirement.txt" :

```
pandas #Data Manipulation library
pytest #Framework for automated testing
```

- **project structure**

```
app-github-action/
├── .github/workflows/python-app.yml
├── src/
│   ├── __init__.py
│   └── maths-operations.py
├── tests
│   ├── __init__.py
│   └── test_operations.py
├── requirements.txt
└── readme.md
```

- **src/math_operations.py**

```
def add(a, b):
    return a + b

def substract(a, b):
    return a - b
```

- src/ - contains Application source code
- __init__.py - Marks a directory as a python package.
- Modular code makes functions easier to reuse & test.

(3.) Unit Testing with Pytest

- **tests/test_operations.py**

```
from src.math_operations import add, subtract

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_subtract():
    assert subtract(5,3) == 2
    assert subtract(4, 3) == 1
    assert subtract(3,3) == 0
    assert subtract(2, 3) == -1
```

- **Unit testing** = Tests individual functions independently.
- **assert** = checks whether the actual result matches the expected result.
- **test_prefix** = Helps pytest discover test functions automatically.

- **For Running Test Locally -**

```
pip install -r requirements.txt
python -m pytest
```

- **For Push changes -**

```
git add .
git commit -m "Add unit tests"
git push origin main
```

(4) GitHub Actions Workflow Configuration

- Workflow files use YAML syntax (.yml) or (.yaml).
- They must be stored inside .github/workflows/.
- create python-app.yml to automate testing.

- .github/workflows/python-app.yml

```
name: Python CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses : actions/checkout@v4

      - name : Set up Python
        uses : actions/setup-python@v5
        with :
          python-version : "3.12"

      - name : Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      -name : Run tests
       run: python -m pytest
```

* Important YAML Keywords

Keyword	Purpose
name	workflow display name
on	Events that trigger the workflow
push	Triggers when commits are pushed to a matching branch.
pull_request	Triggers on matching pull request events
jobs	Defines the jobs to execute
runs-on	specifies the runner operating system
steps	Lists tasks executed sequentially
uses	Executes a reusable Github Action
with	Provides action configuration
run	Executes shell commands.

→ (5.) Workflow Execution

(i) set up Job - Prepares the runner environment.

(ii) checkout code - Downloads the repository files.

(iii) set up Python - Configures the Python runtime.

(iv) Install Dependencies - Install packages from "requirements.txt"

(v) Run Tests - Executes pytest

(vi) cleanup - Performs post-job tasks & finish the job.

↳ Monitoring the Pipeline

(1.) Push the workflow file to Github.

(2.) open the repository's Actions tab.

(3) select the latest workflow run.

(4) Inspect the logs for each step.

- Success = All required steps pass.
- Failure = A step encounters an error or a test fails.

↳ (6) Automated Regression Prevention

- Every matching push or pull request triggers automated testing.
- Failed tests reveal bugs before changes are integrated.
- Required status checks can block a pull request from merging when CI fails.
- This process helps prevent existing functionality from breaking after code changes.